

STM32F7 的 MPU 内存保护单元

本章节为大家讲解 STM32F7 学习中的一个重要知识点 MPU (Memory Protection Unit, 内存保护单元), 早在 STM32F1 和 F4 芯片上面也是有这个功能的, 但是基本用不上。但是到了 H7 就得用了, 因为要设置 Cache。

1.1 初学者重要提示

1.2 MPU 简介

1.3 MPU 的功能实现

1.4 MPU 可以配置的三种内存类型

1.5 MPU 的寄存器和对应的库参数

1.6 MPU 的配置函数

1.7 总结

1.1 初学者重要提示

- ◆ 本章节主要是为下一章 Cache 的讲解做个铺垫, 需要初学者把本章节涉及到的基础知识点掌握了。
- ◆ 初学 MPU 时, 可能有些知识点无法一下子就搞明白, 在后续章节的各种应用后, 会有一个较深的认识。这个知识点基本会贯穿整个教程。

1.2 MPU 简介

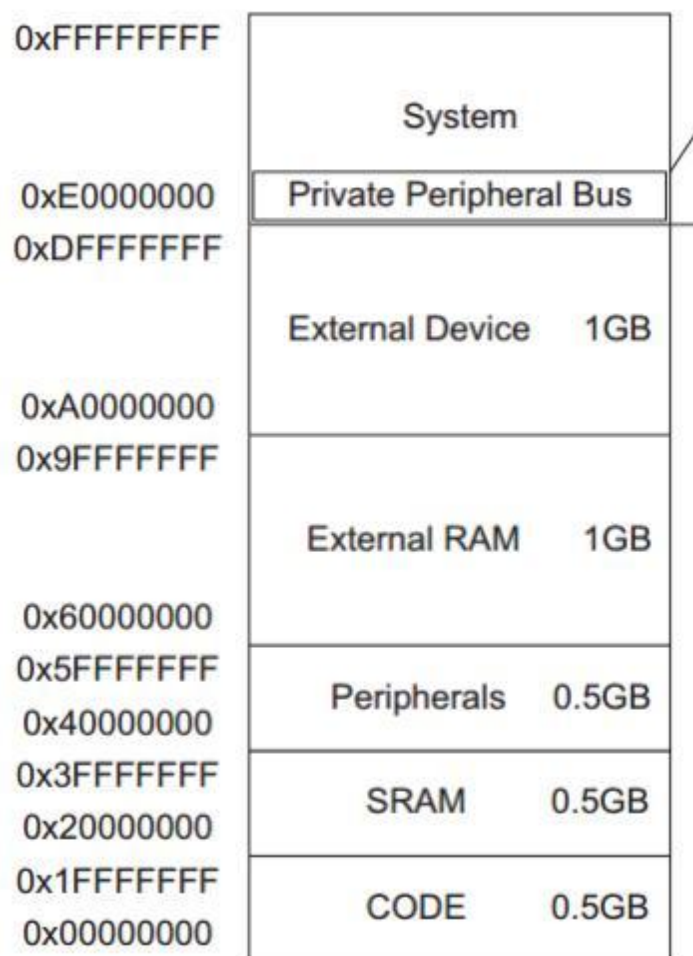
MPU 可以将 memory map 内存映射区分为多个具有一定访问规则的区域, 通过这些规则可以实现如下功能:

- ◆ 防止不受信任的应用程序访问受保护的内存区域。
- ◆ 防止用户应用程序破坏操作系统使用的数据。
- ◆ 通过阻止任务访问其它任务的数据区。
- ◆ 允许将内存区域定义为只读, 以便保护重要数据。
- ◆ 检测意外的内存访问。

简单的说就是内存保护、外设保护和代码访问保护。

- 内存映射

内存映射就是 32 位的 CM7 内核整体可以寻址的 0 到 $2^{32}-1$ 共计 4GB 的寻址空间。通过这些地址可以访问 RAM、Flash、外设等。下面是内存映射的轮廓图，IC 厂家使用时，再做细分，添加相应的硬件功能。

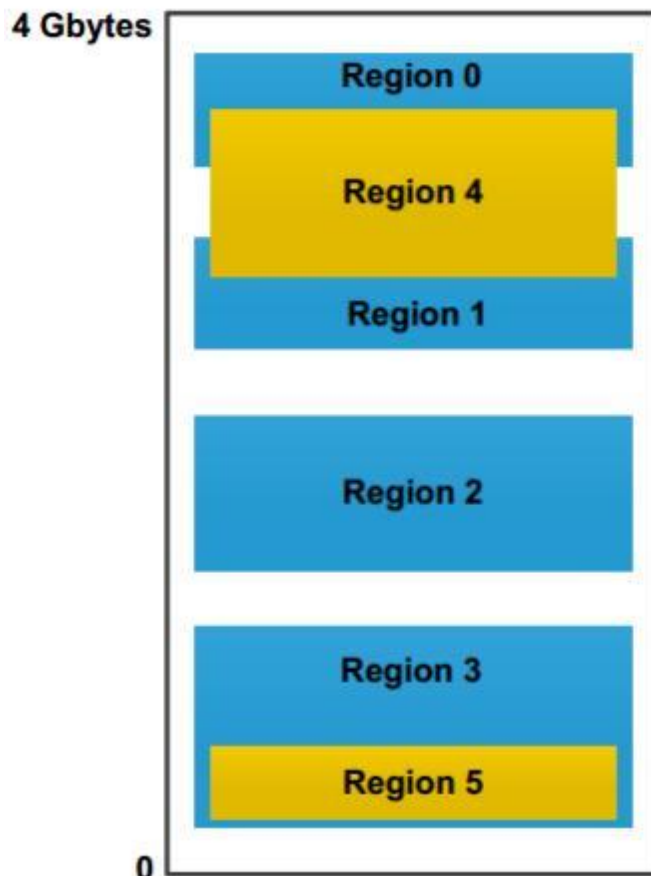


1.3 MPU 的功能实现

MPU 可以配置保护 16 个内存区域（这 16 个内存域是独立配置的），每个区域最小要求 256 字节，每个区域还可以配置为 8 个子区域。由于子区域一般都相同大小，这样每个子区域的大小就是 32 字节，正好跟 Cache 的 Cache Line 大小一样。

MPU 可以配置的 16 个内存区的序号范围是 0 到 15，还有默认区 default region，也叫作背景区，序号-1。由于这些内存区可以嵌套和重叠，所以这些区域在嵌套或者重叠的时候有个优先级的問題。序号 15 的优先级最高，以此递减，序号-1，即背景区的优先级最低。这些优先级是固定的。

下面通过一个具体的实例帮助大家理解。如下所示共有 7 个区，背景区和序号 0-5 的区。内存区 4 跟内存区 0 和 1 有重叠部分，那么重叠部分将按照内存区 4 的配置规则执行；内存区 5 被完全包含在内存区 3 里面，那么这部分内存区将按照内存区 5 的配置规则执行。



1.4 MPU可以配置的三种内存类型

MPU 可以配置的三种内存类型如下：

- ◆ Normal memory

CPU 以最高效的方式加载和存储字节、半字和字，对于这种内存区，CPU 的加载或存储不一定要按照程序列出的顺序执行。

- ◆ Device memory

对于这种类型的内存区，加载和存储要严格按照次序进行，这样是为了确保寄存器按照正确顺序设置。

- ◆ Strongly ordered memory

程序完全按照代码顺序执行，CPU 需要等待当前的加载/存储指令执行完毕后才执行下一条指令。这会导致性能下降。

1.5 MPU 的寄存器和对应的库参数

关于 MPU 的寄存器介绍在 STM32F7 的编程手册有专门的讲解说明，我们这里重点讲解寄存器 MPU_RASR 和控制寄存器，此寄存器的定义如下：

Bits	Name	Function
[31:29]	-	Reserved.
[28]	XN	Instruction access disable bit: 0: Instruction fetches enabled. 1: Instruction fetches disabled.
[27]	-	Reserved.
[26:24]	AP	Access permission field, see Table 93 on page 229 .
[23:22]	-	Reserved.
[21:19, 17, 16]	TEX, C, B	Memory access attributes, see Table 91 on page 228 .
[18]	S	Shareable bit, see Table 91 on page 228 .
[15:8]	SRD	Subregion disable bits. For each bit in this field: 0: Corresponding sub-region is enabled. 1: Corresponding sub-region is disabled. See Subregions on page 231 for more information. Region sizes of 128 bytes and less do not support subregions. When writing the attributes for such a region, write the SRD field as 0x00.
[7:6]	-	Reserved.
[5:1]	SIZE	Specifies the size of the MPU protection region. The minimum permitted value is 4 (0b00100), see SIZE field values for more information.
[0]	ENABLE	Region enable bit.

1.5.1 RASR 寄存器的 XN 位

XN=0 表示使能指令提取，即这块内存区可以执行程序代码，XN=1 表示禁止指令提取，即这块内存区禁止执行程序代码。

对应的 HAL 库 MPU 参数如下：

```

/** @defgroup CORTEX_MPU_Instruction_Access CORTEX MPU Instruction Access
 * @{
 */
#define MPU_INSTRUCTION_ACCESS_ENABLE ((uint8_t)0x00)
#define MPU_INSTRUCTION_ACCESS_DISABLE ((uint8_t)0x01)

```

1.5.2 RASR 寄存器的 AP 位

AP 的具体定义如下：

AP[2:0]	Privileged permissions	Unprivileged permissions	Description
000	No access	No access	All accesses generate a permission fault
001	RW	No access	Access from privileged software only
010	RW	RO	Writes by unprivileged software generate a permission fault
011	RW	RW	Full access
100	Unpredictable	Unpredictable	Reserved
101	RO	No access	Reads by privileged software only
110	RO	RO	Read only, by privileged or unprivileged software
111	RO	RO	Read only, by privileged or unprivileged software

这几个参数对应的 HAL 库 MPU 参数如下：

```

/** @defgroup CORTEX_MPU_Region_Permission_Attributes CORTEX MPU Region Permission Attributes
 * @{
 */
#define MPU_REGION_NO_ACCESS      ((uint8_t)0x00)
#define MPU_REGION_PRIV_RW       ((uint8_t)0x01)
#define MPU_REGION_PRIV_RW_URO   ((uint8_t)0x02)
#define MPU_REGION_FULL_ACCESS   ((uint8_t)0x03)
#define MPU_REGION_PRIV_RO       ((uint8_t)0x05)
#define MPU_REGION_PRIV_RO_URO   ((uint8_t)0x06)

```

1.5.3 RASR 寄存器的 TEX , C , B 和 S 位

(注， 这几个位非常重要，当前先了解知识点即可，下个章节专门讲解具体的作用)

TEX , C , B 和 S 的定义如下，这仅关注 TEX = 0b000 和 0b001，其它的 TEX 配置基本用不到。

TEX	C	B	S	Memory type	Shareability	Other attributes
0b000	0	0	x ⁽¹⁾	Strongly-ordered	Shareable	-
		1	x ⁽¹⁾	Device	Shareable	-
	1	0	0	Normal	Not shareable	Outer and inner write-through. No write allocate.
			1		Shareable	
		1	0	Normal	Not shareable	Outer and inner write-back. No write allocate.
			1		Shareable	
0b001	0	0	0	Normal	Not shareable	Outer and inner noncacheable.
			1		Shareable	
		1	x ⁽¹⁾	Reserved encoding		-
	1	0	x ⁽¹⁾	Implementation defined attributes.		-
		1	0	Normal	Not shareable	Outer and inner write-back. Write and read allocate.
			1		Shareable	

TEX 用于配置 Cache 策略，支持如下四种情况，需要配合 C 和 B 位的配置才能实现。

Corresponding cache policy
Non-cacheable
Write back, write and read allocate
Write through, no write allocate
Write back, no write allocate

TEX 对应的 HAL 库 MPU 参数给了三个，实际应用中仅用到前两个 MPU_TEX_LEVEL0 和 MPU_TEX_LEVEL1

```

/** @defgroup CORTEX_MPU_TEX_Levels MPU TEX Levels
 * @{
 */
#define MPU_TEX_LEVEL0 ((uint8_t)0x00)
#define MPU_TEX_LEVEL1 ((uint8_t)0x01)
#define MPU_TEX_LEVEL2 ((uint8_t)0x02)

```

C 位对应的 HAL 库 MPU 参数如下，用于使能或者禁止 Cache。

```

/** @defgroup CORTEX_MPU_Access_Cacheable CORTEX MPU Instruction Access Cacheable
 * @{
 */
#define MPU_ACCESS_CACHEABLE ((uint8_t)0x01)
#define MPU_ACCESS_NOT_CACHEABLE ((uint8_t)0x00)

```

B 位对应的 HAL 库 MPU 参数如下，用于配合 C 位实现 Cache 模式下是否使用缓冲。

```

/** @defgroup CORTEX_MPU_Access_Bufferable CORTEX MPU Instruction Access Bufferable
 * @{

```

```

*/
#define MPU_ACCESS_BUFFERABLE          ((uint8_t)0x01)
#define MPU_ACCESS_NOT_BUFFERABLE      ((uint8_t)0x00)

S 位对应的 HAL 库 MPU 参数如下， 用于解决多总线或者多核访问的共享问题。
/** @defgroup CORTEX_MPU_Access_Shareable CORTEX MPU Instruction Access Shareable
 * @{
 */
#define MPU_ACCESS_SHAREABLE           ((uint8_t)0x01)
#define MPU_ACCESS_NOT_SHAREABLE       ((uint8_t)0x00)

```

1.5.4 RASR 寄存器的 SRD 位

这个位用于控制内存区的子区域，使用的是 bit[15:8]，共计 8 个 bit，一个 bit 控制一个子区域，0 表示使能此子区域，1 表示禁止此子区域。

一般情况，基本不使用子区域的禁止功能，所以配置 HAL 库的 SubRegionDisable 参数时，直接取值 0x00 即可，表示 8 个子区域均使能。

1.5.5 RASR 寄存器的 SIZE 位

SIZE 位使用的是 bit[5:1]，共计 5 个 bit，可以表示 $2^5 = 32$ 种大小。 对应的 HAL 库给出了可以配置的 28 个参数：

```

/** @defgroup CORTEX_MPU_Region_Size CORTEX MPU Region Size
 * @{
 */
#define MPU_REGION_SIZE_32B            ((uint8_t)0x04)
#define MPU_REGION_SIZE_64B            ((uint8_t)0x05)
#define MPU_REGION_SIZE_128B           ((uint8_t)0x06)
#define MPU_REGION_SIZE_256B           ((uint8_t)0x07)
#define MPU_REGION_SIZE_512B           ((uint8_t)0x08)
#define MPU_REGION_SIZE_1KB            ((uint8_t)0x09)
#define MPU_REGION_SIZE_2KB            ((uint8_t)0x0A)
#define MPU_REGION_SIZE_4KB            ((uint8_t)0x0B)
#define MPU_REGION_SIZE_8KB            ((uint8_t)0x0C)
#define MPU_REGION_SIZE_16KB           ((uint8_t)0x0D)
#define MPU_REGION_SIZE_32KB           ((uint8_t)0x0E)
#define MPU_REGION_SIZE_64KB           ((uint8_t)0x0F)
#define MPU_REGION_SIZE_128KB          ((uint8_t)0x10)
#define MPU_REGION_SIZE_256KB          ((uint8_t)0x11)
#define MPU_REGION_SIZE_512KB          ((uint8_t)0x12)
#define MPU_REGION_SIZE_1MB            ((uint8_t)0x13)
#define MPU_REGION_SIZE_2MB            ((uint8_t)0x14)
#define MPU_REGION_SIZE_4MB            ((uint8_t)0x15)
#define MPU_REGION_SIZE_8MB            ((uint8_t)0x16)
#define MPU_REGION_SIZE_16MB           ((uint8_t)0x17)
#define MPU_REGION_SIZE_32MB           ((uint8_t)0x18)
#define MPU_REGION_SIZE_64MB           ((uint8_t)0x19)
#define MPU_REGION_SIZE_128MB          ((uint8_t)0x1A)
#define MPU_REGION_SIZE_256MB          ((uint8_t)0x1B)
#define MPU_REGION_SIZE_512MB          ((uint8_t)0x1C)

```

```
#define MPU_REGION_SIZE_1GB    ((uint8_t)0x1D)
#define MPU_REGION_SIZE_2GB    ((uint8_t)0x1E)
#define MPU_REGION_SIZE_4GB    ((uint8_t)0x1F)
```

1.5.6 CTRL 寄存器的各个位

控制寄存器各个位定义如下如下：

Bits	Name	Function
[31:3]	-	Reserved.
[2]	PRIVDEFENA	Enables privileged software access to the default memory map: 0: If the MPU is enabled, disables use of the default memory map. Any memory access to a location not covered by any enabled region causes a fault. 1: If the MPU is enabled, enables use of the default memory map as a background region for privileged software accesses. When enabled, the background region acts as if it is region number -1. Any region that is defined and enabled has priority over this default map. If the MPU is disabled, the processor ignores this bit.
[1]	HFNMENA	Enables the operation of MPU during hard fault, NMI, and FAULTMASK handlers. When the MPU is enabled: 0: MPU is disabled during hard fault, NMI, and FAULTMASK handlers, regardless of the value of the ENABLE bit. 1: The MPU is enabled during hard fault, NMI, and FAULTMASK handlers. When the MPU is disabled, if this bit is set to 1 the behavior is Unpredictable.
[0]	ENABLE	Enables the MPU: 0: MPU disabled. 1: MPU enabled.

1.6 MPU 的配置函数

HAL 库的 stm32f7xx_hal_cortex.c 文件为 MPU 的配置提供了三个函数：

- ◆ HAL_MPU_Disable
- ◆ HAL_MPU_Enable
- ◆ HAL_MPU_ConfigRegion

三个函数的使用都比较简单，但是要让配置的内存区最大限度的发挥性能是需要大量的经验积累和测试的。具体工程要具体分析。

1.6.1 函数 HAL_MPU_Disable

函数原型:

```
void HAL_MPU_Disable(void)
{
    /* Make sure outstanding transfers are done */
    __DMB();

    /* Disable fault exceptions */
    SCB->SHCSR &= ~SCB_SHCSR_MEMFAULTENA_Msk;

    /* Disable the MPU and clear the control register*/
    MPU->CTRL = 0;
}
```

函数描述:

此函数用于禁止 MPU。配置 MPU 前要优先调用此函数禁止 MPU，然后才可以配置。

1.6.2 函数 HAL_MPU_Enable

函数原型:

```
void HAL_MPU_Enable(uint32_t MPU_Control)
{
    /* Enable the MPU */
    MPU->CTRL = MPU_Control | MPU_CTRL_ENABLE_Msk;

    /* Enable fault exceptions */
    SCB->SHCSR |= SCB_SHCSR_MEMFAULTENA_Msk;

    /* Ensure MPU setting take effects */
    __DSB();
    __ISB();
}
```

函数描述:

此函数用于使能 MPU，一般情况使用参数 MPU_PRIVILEGED_DEFAULT。

函数参数:

此函数支持以下几个参数:

- ◆ **MPU_HFNMI_PRIVDEF_NONE** ((uint32_t)0x00000000)
 - 此参数设置 MPU 的 CTL 控制寄存器的 PRIVDEFENA 位为 0。
表示禁止了背景区，访问任何未使能 MPU 的区域均会造成内存异常 MemFault。
 - 此参数设置 MPU 的 CTL 控制寄存器的 HFNMIENA 位为 0。
表示 NMI 不可屏蔽中断服务程序和硬件异常中断服务程序执行期间会关闭 MPU。
- ◆ **MPU_HARDFAULT_NMI** ((uint32_t)0x00000002)
 - 此参数设置 MPU 的 CTL 控制寄存器的 PRIVDEFENA 位为 0。
表示使能了背景区，访问任何未使能 MPU 的区域均会造成内存异常 MemFault。
 - 此参数设置 MPU 的 CTL 控制寄存器的 HFNMIENA 位为 1。
表示 NMI 不可屏蔽中断服务程序和硬件异常中断服务程序执行期间会保持继续开启 MPU。

◆ MPU_PRIVILEGED_DEFAULT ((uint32_t)0x00000004)

- 此参数设置 MPU 的 CTL 控制寄存器的 PRIVDEFENA 位为 1。
表示使能了背景区， 特权级模式可以正常访问任何未使能 MPU 的区域。
- 此参数设置 MPU 的 CTL 控制寄存器的 HFNMIENA 位为 0。
表示 NMI 不可屏蔽中断服务程序和硬件异常中断服务程序执行期间会关闭 MPU。

◆ MPU_HFNMI_PRIVDEF ((uint32_t)0x00000006)

- 此参数设置 MPU 的 CTL 控制寄存器的 PRIVDEFENA 位为 1。
表示禁止了背景区， 访问任何未使能 MPU 的区域均会造成内存异常 MemFault。
- 此参数设置 MPU 的 CTL 控制寄存器的 HFNMIENA 位为 1。
表示 NMI 不可屏蔽中断服务程序和硬件异常中断服务程序执行期间会保持继续开启 MPU。

1.6.3 函数 HAL_MPU_ConfigRegion

函数原型：

```

void HAL_MPU_ConfigRegion(MPU_Region_InitTypeDef *MPU_Init)
{
    /* Check the parameters */
    assert_param(IS_MPU_REGION_NUMBER(MPU_Init->Number));
    assert_param(IS_MPU_REGION_ENABLE(MPU_Init->Enable));

    /* Set the Region number */
    MPU->RNR = MPU_Init->Number;

    if ((MPU_Init->Enable) != RESET)
    {
        /* Check the parameters */
        assert_param(IS_MPU_INSTRUCTION_ACCESS(MPU_Init->DisableExec));
        assert_param(IS_MPU_REGION_PERMISSION_ATTRIBUTE(MPU_Init->AccessPermission));
        assert_param(IS_MPU_TEX_LEVEL(MPU_Init->TypeExtField));
        assert_param(IS_MPU_ACCESS_SHAREABLE(MPU_Init->IsShareable));
        assert_param(IS_MPU_ACCESS_CACHEABLE(MPU_Init->IsCacheable));
        assert_param(IS_MPU_ACCESS_BUFFERABLE(MPU_Init->IsBufferable));
        assert_param(IS_MPU_SUB_REGION_DISABLE(MPU_Init->SubRegionDisable));
        assert_param(IS_MPU_REGION_SIZE(MPU_Init->Size));

        MPU->RRBAR = MPU_Init->BaseAddress;

        /* RASR configuration */
        ((uint32_t)MPU_Init->AccessPermission << MPU_RASR_AP_Pos) |
        ((uint32_t)MPU_Init->TypeExtField << MPU_RASR_TEX_Pos) |
        ((uint32_t)MPU_Init->IsShareable << MPU_RASR_S_Pos) |
        ((uint32_t)MPU_Init->IsCacheable << MPU_RASR_C_Pos) |
        ((uint32_t)MPU_Init->IsBufferable << MPU_RASR_B_Pos) |
        ((uint32_t)MPU_Init->SubRegionDisable << MPU_RASR_SRD_Pos) |
        ((uint32_t)MPU_Init->Size << MPU_RASR_SIZE_Pos) |
        ((uint32_t)MPU_Init->Enable << MPU_RASR_ENABLE_Pos);
    }
    else
    {
        MPU->RBAR = 0x00;
        MPU->RASR = 0x00;
    }
}

```

函数描述：

此函数用于配置 MPU。

函数参数：

此函数的形参是一个 MPU_Region_InitTypeDef 类型的结构体变量，定义如下：

```

typedef struct
{
    uint8_t      Enable;
    uint8_t      Number;
    uint32_t     BaseAddress;
    uint8_t      Size;
    uint8_t      SubRegionDisable;
    uint8_t      TypeExtField;
    uint8_t      AccessPermission;
    uint8_t      DisableExec;
    uint8_t      IsShareable;
    uint8_t      IsCacheable;
    uint8_t

```

```
IsBufferable;  
}MPU_Region_InitTypeDef;
```

除了参数 Number 和 BaseAddress，其它几个参数在本章 1.5 小节进行了说明。

◆ 结构体成员 Number

这个成员是用来设置内存区序号的，用户配置的时候，推荐从 Number0 开始配置，最多到 Number15，共计 16 个。对应的 HAL 库参数如下：

```
/** @defgroup CORTEX_MPU_Region_Number CORTEX MPU Region Number  
 * @{  
 */  
#define MPU_REGION_NUMBER0    ((uint8_t)0x00)  
#define MPU_REGION_NUMBER1    ((uint8_t)0x01)  
#define MPU_REGION_NUMBER2    ((uint8_t)0x02)  
#define MPU_REGION_NUMBER3    ((uint8_t)0x03)  
#define MPU_REGION_NUMBER4    ((uint8_t)0x04)  
#define MPU_REGION_NUMBER5    ((uint8_t)0x05)  
#define MPU_REGION_NUMBER6    ((uint8_t)0x06)  
#define MPU_REGION_NUMBER7    ((uint8_t)0x07)  
#define MPU_REGION_NUMBER8    ((uint8_t)0x08)  
#define MPU_REGION_NUMBER9    ((uint8_t)0x09)  
#define MPU_REGION_NUMBER10   ((uint8_t)0x0A)  
#define MPU_REGION_NUMBER11   ((uint8_t)0x0B)  
#define MPU_REGION_NUMBER12   ((uint8_t)0x0C)  
#define MPU_REGION_NUMBER13   ((uint8_t)0x0D)  
#define MPU_REGION_NUMBER14   ((uint8_t)0x0E)  
#define MPU_REGION_NUMBER15   ((uint8_t)0x0F)
```

◆ 结构体成员 BaseAddress

这个结构体成员用来设置内存区的首地址。这个参数跟结构体成员 Size 的配置是比较考究的，一定要保证首地址跟内存区的大小对齐，比如配置的是 64KB 大小的内存区，那么设置的首地址必须是 64KB 对齐的，也就是这个地址对 64KB，即 0x00010000 求余数等于 0，切记。

使用举例：

下面配置了两组。

```
/*  
*****  
 * 函数名：MPU_Config  
 * 功能说明：配置 MPU  
 * 形参：无  
 * 返回值：无  
*****  
*/  
static void MPU_Config( void )  
{  
    MPU_Region_InitTypeDef MPU_InitStruct;  
  
    /* 禁止 MPU */  
    HAL_MPU_Disable();  
  
    /* 配置 AXI SRAM 的 MPU 属性为 Write back, Read allocate, Write allocate */  
    MPU_InitStruct.Enable      MPU_InitStruct.BaseAddress MPU_InitStruct.Size
```

```

= MPU_REGION_ENABLE;
= 0x24000000;
= MPU_REGION_SIZE_512KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable      = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable       = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable       = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number            = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField       = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable  = 0x00;
    MPU_InitStruct.DisableExec       = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /* 配置 FMC 扩展 IO 的 MPU 属性为 Device 或者 Strongly Ordered */
    MPU_InitStruct.Enable            MPU_InitStruct.BaseAddress MPU_InitStruct.Size

```

```

= MPU_REGION_ENABLE;
= 0x60000000;
= ARM_MPU_REGION_SIZE_64KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable      = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable       = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable       = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number            = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField       = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0x00;
    MPU_InitStruct.DisableExec       = MPU_INSTRUCTION_ACCESS_ENABLE;

    HAL_MPU_ConfigRegion(&MPU_InitStruct);

    /*使能 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

```

1.7 总结

本章节就为大家讲解这么多， 还是本章开头那句话，可能有些知识点无法一下子就搞明白， 在学习了后续章节的各种应用后， 会有一个较深的认识。

STM32F7 的 Cache解读

本章教程为大家讲解 STM32F7 初学过程中最重要的一个知识点Cache。Cache 在 STM32F7 的性能发挥中占着举足轻重的作用。所以掌握好 Cache 是提升 STM32F7 性能的关键一步。

2.1 初学者重要提示

2.2 引出问题

2.3 支持的 Cache 配置

2.4 四种 Cache (MPU) 配置的读写操作流程

2.5 面对繁冗复杂的 Cache 配置， 推荐方式和安全隐患解决办法

2.6 Cache 的相关函数

2.7 总结

2.1初学者重要提示

- ◆ 学习本章节前， 务必保证已经学习了第 1 章的 MPU 知识。
- ◆ 本章是半年的实践经验总结， 非常具有参考价值， 而且是入门STM32F7 的必学章节。

- ◆ Cache 的熟练运用需要不断的经验积累。对于初学者来说，可能无法一下子理解所有知识点，但是一定要花时间去多读几遍，随着后面章节的不断运用，认识会不断的深入。

2.2引出问题

当前芯片厂商出的 M7 内核芯片基本都做了一级 Cache 支持，Cache 又分数据缓存 D-Cache 和指令缓冲 I-Cache，STM32F7 的数据缓存和指令缓存大小都是 16KB。对于指令缓冲，如果不在 RAM 里运行程序，用户不用管，这里主要说的是数据缓存 D-Cache。数据缓存 D-Cache 就是解决 CPU 加速访问 SRAM。

如果每次 CPU 要读写 SRAM 区的数据，都能够在 Cache 里面进行，自然是最好的，实际是做不到的，因为数据 Cache 只有 16KB 大小，总有用完的时候。

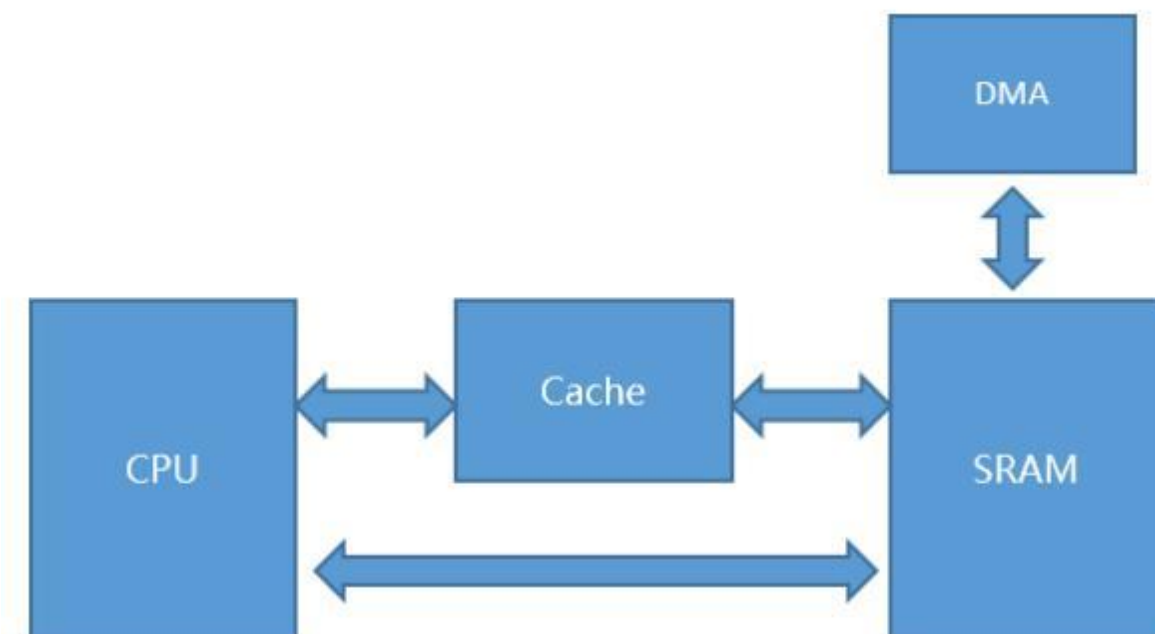
对于使能了 Cache 的 SRAM 区，要分读写两种情况考虑。

◆ 读操作：

如果 CPU 要读取的 SRAM 区数据在 Cache 中已经加载好，这就叫读命中(Cache hit)，如果 Cache 里面没有怎么办，这就是所谓的读 Cache Miss。

◆ 写操作：

如果 CPU 要写的 SRAM 区数据在 Cache 中已经开辟了对应的区域(专业词汇叫 Cache Line，以 32 字节为单位)，这就叫写命中(Cache hit)，如果 Cache 里面没有开辟对应的区域怎么办，这就是所谓的写 Cache Miss。



2.3支持的Cache配置

(这个知识点在上一章节进行了详细说明，这里再简述下核心内容)

Cache 的配置是通过 MPU 来设置的，通常只用到下几种方式。

Table 91. TEX, C, B, and S encoding

TEX	C	B	S	Memory type	Shareability	Other attributes
0b000	0	0	x ⁽¹⁾	Strongly-ordered	Shareable	-
		1	x ⁽¹⁾	Device	Shareable	-
	1	0	0	Normal	Not shareable	Outer and inner write-through. No write allocate.
			1		Shareable	
		1	0	Normal	Not shareable	Outer and inner write-back. No write allocate.
			1		Shareable	
0b001	0	0	0	Normal	Not shareable	Outer and inner noncacheable.
			1		Shareable	
		1	x ⁽¹⁾	Reserved encoding		-
	1	0	x ⁽¹⁾	Implementation defined attributes.		-
		1	0	Normal	Not shareable	Outer and inner write-back. Write and read allocate.
			1		Shareable	

其中的 TEX 是用来设置 Cache 策略的，C 是 Cache，B 是缓冲用来配合 Cache 设置的，而 S 是共享，用来解决多总线或者多核访问时的同步问题。MPU 配置的时候，最主要的也是配置这几个参数。

Cache 支持的策略有如下四种：

Corresponding cache policy
Non-cacheable
Write back, write and read allocate
Write through, no write allocate
Write back, no write allocate

有了这四种方式，就可以正式进入本章的主题，Cache 的读写操作是如何工作的，下面分这四种情况做介绍。

2.4 四种 Cache (MPU) 配置的读写操作流程

2.4.1 配置 Non-cacheable

这个最好理解，就是正常的读写操作，无 Cache。

对应四种 MPU 配置如下：

- ◆ TEX = 000 C=0 B=0 S=忽略此位，强制为共享
- ◆ TEX = 000 C=0 B=1 S=忽略此位，强制为共享
- ◆ TEX = 001 C=0 B=0 S=0
- ◆ TEX = 001 C=0 B=0 S=1

2.4.2 配置 Write through , read allocate , no write allocate

注意，M7 内核只要开启了 Cache，read allocate 就是开启的。

◆ 使能了此配置的 SRAM 缓冲区写操作

如果 CPU 要写的 SRAM 区数据在 Cache 中已经开辟了对应的区域，那么会同时写到 Cache 里面和 SRAM 里面；如果没有，就用到配置 no write allocate 了，意思就是 CPU 会直接往 SRAM 里面写数据，而不再需要在 Cache 里面开辟空间了。

在写 Cache 命中的情况下，这个方式的优点是 Cache 和 SRAM 的数据同步更新了，没有多总线访问造成的数据一致性问题。缺点也明显，Cache 在写操作上无法有效发挥性能。

◆ 使能了此配置的 SRAM 缓冲区读操作

如果 CPU 要读取的 SRAM 区数据在 Cache 中已经加载好，就可以直接从 Cache 里面读取。如果没有，就用到配置 read allocate 了，意思就是在 Cache 里面开辟区域，将 SRAM 区数据加载进来，后续的操作，CPU 可以直接从 Cache 里面读取，从而时间加速。

安全隐患，如果 Cache 命中的情况下，DMA 写操作也更新了 SRAM 区的数据，CPU 直接从 Cache 里面读取的数据就是错误的。

◆ 对应的两种 MPU 配置如下：

TEX = 000 C=1 B=0 S=1

TEX = 000 C=1 B=0 S=0

2.4.3 配置 Write back , read allocate , no write allocate

注意，M7 内核只要开启了 Cache，read allocate 就是开启的。

◆ 使能了此配置的 SRAM 缓冲区写操作

如果 CPU 要写的 SRAM 区数据在 Cache 中已经开辟了对应的区域，那么会写到 Cache 里面，而不会立即更新 SRAM；如果没有，就用到配置 no write allocate 了，意思就是 CPU 会直接往 SRAM 里面写数据，而不再需要在 Cache 里面开辟空间了。

安全隐患，如果 Cache 命中的情况下，此时仅 Cache 更新了，而 SRAM 没有更新，那么 DMA 直接从 SRAM 里面读出来的就是错误的。

◆ 使能了此配置的 SRAM 缓冲区读操作

如果 CPU 要读取的 SRAM 区数据在 Cache 中已经加载好，就可以直接从 Cache 里面读取。如果没有，就用到配置 read allocate 了，意思就是在 Cache 里面开辟区域，将 SRAM 区数据加载进来，后续的操作，CPU 可以直接从 Cache 里面读取，从而时间加速。

安全隐患，如果 Cache 命中的情况下，DMA 写操作也更新了 SRAM 区的数据，CPU 直接从 Cache 里面读取的数据就是错误的。

◆ 对应两种 MPU 配置如下：

TEX = 000 C=1 B=1 S=1

TEX = 000 C=1 B=1 S=0

2.4.4配置 Write back , read allocate , write allocate

注意，M7 内核只要开启了 Cache，read allocate 就是开启的。

◆ 使能了此配置的 SRAM 缓冲区写操作

如果 CPU 要写的 SRAM 区数据在 Cache 中已经开辟了对应的区域，那么会写到 Cache 里面，而不会立即更新 SRAM；如果没有，就用到配置 write allocate 了，意思就是 CPU 写到往 SRAM 里面的数据，会同步在 Cache 里面开辟一个空间将 SRAM 中写入的数据加载进来，如果此时立即读此 SRAM 区，那么就会有很大的速度优势。

安全隐患，如果 Cache 命中的情况下，此时仅 Cache 更新了，而 SRAM 没有更新，那么 DMA 直接从 SRAM 里面读出来的就是错误的。

◆ 使能了此配置的 SRAM 缓冲区读操作

如果 CPU 要读取的 SRAM 区数据在 Cache 中已经加载好，就可以直接从 Cache 里面读取。如果没有，就用到配置 read allocate 了，意思就是在 Cache 里面开辟区域，将 SRAM 区数据加载进来，后续的操作，CPU 可以直接从 Cache 里面读取，从而时间加速。

安全隐患，如果 Cache 命中的情况下，DMA 写操作也更新了 SRAM 区的数据，CPU 直接从 Cache 里面读取的数据就是错误的。

这个配置被誉为可以最大程度发挥 Cache 性能，不过具体应用仍需具体分析。

◆ 对应两种 MPU 配置如下：

TEX = 001 C=1 B=1 S=1

TEX = 001 C=1 B=1 S=0

2.4.5 共享配置是个隐形的大坑

STM32F7 编程手册对其的描述是多核共享。

Table 83. Memory attributes summary (continued)

Memory type	Shareability	Other attributes	Description
Normal	Shared	Non-cacheable Write-through Cacheable Write-back Cacheable	Normal memory that is shared between several processors.
-	Non-shared	Non-cacheable Write-through Cacheable Write-back Cacheable	Normal memory that only a single processor uses.

而 F7 的应用笔记对齐的描述是开启共享基本等同于关闭 Cache。

The S field is for a shareable memory region: the memory system provides data synchronization between bus masters in a system with multiple bus masters, for example, a processor with a DMA controller. Strongly-ordered memory is always shareable. If multiple bus masters can access a non-shareable memory region, the software must ensure the data coherency between the bus masters. The STM32F7 Series and STM32H7 Series do not support hardware coherency. the S field is equivalent to non-cacheable memory.

实际测试下面四种开 Cache 的情况，开关共享对缓冲区的大批量数据的读操作影响很大，基本差出两倍，而写操作基本没有影响，也许这就是所谓的多总线同步读造成的。另外共享开关仅对开启了Cache 的情况下有影响，而对于关闭了 Cache 的情况是没有影响的，开不开没关系。

2.4.6 总结这几种方式的几个关键知识点

1. Cortex-M7 内核的 L1 Cache 由多行内存区组成，每行有 32 字节，每行都配有一个地址标签。数据缓冲 DCache 是每 4 行为一组，称为 4-way set associative。而指令缓冲区 ICache 是 2 行为一组，这样节省地址标签，不用每个行都标记一个地址。
2. 对于读操作，只有在第 1 次访问指定地址时才会加载到 Cache，而写操作的话，可以直接写到内存中 (write-through 模式) 或者放到 Cache 里面，后面再写入(write-back 模式)。
3. 如果采用的是 Write back，Cache line 会被标为 dirty，等到此行被 evicted 时，才会执行实际的写操作，将 Cache Line 里面的数据写入到相应的存储区。
4. Cache 命中是访问的地址落在了给定的 Cache Line 里面，所以硬件需要做少量的地址比较工作，以检查此地址是否被缓存。如果命中了，将用于缓存读操作或者写操作。如果没有命中，则分配和标记新行，填充新的读写操作。如果所有行都分配完毕了，Cache 控制器将支持 eviction 操作。根据

Cache Line 替换算法，一行将被清除 Clean，无效化 Invalid 或者重新配置。数据缓存和指令缓存是采用的 伪随机替换算法。

5. Cache 支持的 4 种基本操作，使能，禁止，清空和无效化。Clean 清空操作是将 Cache Line 中标记为 dirty 的数据写入到内存里面，而无效化 Invalid 是将 Cache Line 标记为无效，即删除操作。

2.5 面对繁冗复杂的 Cache 配置，推荐方式和安全隐患解决办法

- ◆ 推荐使用 128KB 的 TCM 作为主 RAM 区，其它的专门用于大缓冲和 DMA 操作等。
- ◆ Cache 问题主要是 CPU 和 DMA 都操作这个缓冲区时容易出现，使用时要注意。
- ◆ Cache 配置的选择，优先考虑的是 WB，然后是 WT 和关闭 Cache，其中 WB 和 WT 的使用中可以配合 ARM 提供的函数解决上面说到的隐患问题 (见本章 2.6 小节)。但不是万能的，在不起作用的时候，直接暴力选择函数 SCB_CleanInvaldateDCache 解决。关于这个问题，在分别配置以太网 MAC 的描述符缓冲区，发送缓冲区和接收缓冲区时尤其突出。

2.6 Cache 的相关函数

CMSIS 软件包的 core_cm7.h 文件为 Cache 的配置提供了 11 个函数：

- ◆ SCB_EnableICache
- ◆ SCB_DisableICache
- ◆ SCB_InvalidateICache
- ◆ SCB_EnableDCache
- ◆ SCB_DisableDCache
- ◆ SCB_InvalidateDCache
- ◆ SCB_CleanDCache
- ◆ SCB_CleanInvalidateDCache
- ◆ SCB_InvalidateDCache_by_Addr
- ◆ SCB_CleanDCache_by_Addr
- ◆ SCB_CleanInvalidateDCache_by_Addr

下面将这几个函数依次做个讲解。其中前三个函数是指令 Cache，比较容易掌握。重点是后面几个数据 Cache 函数。由于函数 SCB_CleanInvalidateDCache，SCB_CleanDCache 和 SCB_InvalidateDCache 是对整个 Cache 的操作，所以比最后的三个函数 SCB_InvalidateDCache_by_Addr，SCB_CleanDCache_by_Addr 和 SCB_CleanInvalidateDCache_by_Addr 要耗时，当然，如果用户操作的存储器超过了数据 Cache 的大小，即 16KB，那么就跟前三个函数没有区别了。

2.6.1 函数 SCB_EnableICache

函数原型：

```
__STATIC_INLINE void SCB_EnableICache (void)
{
    #if defined (__ICACHE_PRESENT) && (__ICACHE_PRESENT == 1U)
        __DSB();
        __ISB();
        SCB->ICIAALLU = 0UL; /* invalidate I-Cache */
        __DSB();
        __ISB();
        SCB->CCR |= (uint32_t)SCB_CCR_IC_Msk; /* enable I-Cache */
        __DSB();
        __ISB();
    #endif
}
```

函数描述：

此函数用于使能指令Cache，系统上电后优先初始化即可。

注意事项：

- ◆ __STATIC_INLINE:
表示内联函数，这种类型函数的作用就是将函数直接嵌入到调用此函数的代码中，从而降低调用此函数所占用的时间。
- ◆ __DMB 指令：
Data Memory Barrier（数据存储器隔离），DMB 指令保证所有在它前面的存储器访问操作都执行完毕后，才提交在它后面的存储器访问操作。
- ◆ __DSB 指令：
Data Synchronization Barrier（数据同步隔离），比 DMB 严格，当所有在它前面的存储器访问操作都执行完毕后，才执行在它后面的指令。
- ◆ __ISB 指令：
Instruction Synchronization Barrier（指令同步隔离），它会清洗流水线，以保证所有它前面的指令都执行完毕之后，才执行它后面的指令。

2.6.2 函数 SCB_DisableICache

函数原型：

```
__STATIC_INLINE void SCB_DisableICache (void)
{
    #if defined (__ICACHE_PRESENT) && (__ICACHE_PRESENT == 1U)
        __DSB();
        __ISB();
        SCB->CCR &= ~(uint32_t)SCB_CCR_IC_Msk; /* disable I-Cache */
        SCB->ICIAALLU = 0UL; /* invalidate I-Cache */
        __DSB();
        __ISB();
    #endif
}
```

函数描述：

此函数用于禁止指令Cache。

注意事项：

__STATIC_INLINE , __DMB , __DSB 和 __ISB 的作用看本章 2.6.1 小节的说明。

2.6.3 函数 SCB_InvalidateICache

函数原型：

```
__STATIC_INLINE void SCB_InvalidateICache (void)
{
    #if defined (__ICACHE_PRESENT) && (__ICACHE_PRESENT == 1U)
        __DSB();
        __ISB();
        SCB->ICIALLU = 0UL;
        __DSB();
        __ISB();
    #endif
}
```

函数描述：

此函数用于将指令 Cache 无效化，无效化的意思是将 Cache Line 标记为无效，等同于删除操作。这样 Cache 空间就都腾出来了，可以加载新的指令。

注意事项：

__STATIC_INLINE , __DMB , __DSB 和 __ISB 的作用看本章 2.6.1 小节的说明。

2.6.4 函数 SCB_EnableDCache

函数原型：

```

__STATIC_INLINE void SCB_EnabledDCache (void)
{
    #if defined (__DCACHE_PRESENT) && (__DCACHE_PRESENT == 1U)
        uint32_t ccsidr;
        uint32_t sets;
        uint32_t ways;

        SCB->CSSELR = 0U; /*(0U << 1U) | 0U;*/ /* Level 1 data cache */
        __DSB();

        ccsidr = SCB->CCSIDR;

        /* invalidate D-Cache */
        sets = (uint32_t)(CCSIDR_SETS(ccsidr));
        do {
            ways = (uint32_t)(CCSIDR_WAYS(ccsidr));
            do {
                SCB->DCISW = (((sets << SCB_DCISW_SET_Pos) & SCB_DCISW_SET_Msk) |
                               ((ways << SCB_DCISW_WAY_Pos) & SCB_DCISW_WAY_Msk) );
                #if defined (__CC_ARM)
                    __schedule_barrier();
                #endif
            } while (ways-- != 0U);
        } while(sets-- != 0U);
        __DSB();

        SCB->CCR |= (uint32_t)SCB_CCR_DC_Msk; /* enable D-Cache */

        __DSB();
        __ISB();
    #endif
}

```

函数描述：

此函数用于使能数据Cache，系统上电后优先初始化即可。

注意事项：

__STATIC_INLINE，__DMB，__DSB 和 __ISB 的作用看本章 2.6.1 小节的说明。

2.6.5 函数 SCB_DisableDCache

函数原型：

```

__STATIC_INLINE void SCB_DisableDCache (void)
{
    #if defined (__DCACHE_PRESENT) && (__DCACHE_PRESENT == 1U)
        register uint32_t ccsidr;
        register uint32_t sets;
        register uint32_t ways;

        SCB->CSSELR = 0U; /*(0U << 1U) | 0U;*/ /* Level 1 data cache */
        __DSB();

        SCB->CCR &= ~(uint32_t)SCB_CCR_DC_Msk; /* disable D-Cache */
        __DSB();
    #endif
}

```

```

ccsidr = SCB->CCSIDR;

/* clean & invalidate D-Cache */
sets = (uint32_t) (CCSIDR_SETS(ccsidr));
do {
    ways = (uint32_t) (CCSIDR_WAYS(ccsidr));
    do {
        SCB->DCCISW = (((sets << SCB_DCCISW_SET_Pos) & SCB_DCCISW_SET_Msk) |
                      ((ways << SCB_DCCISW_WAY_Pos) & SCB_DCCISW_WAY_Msk) );
        #if defined ( __CC_ARM )
            __schedule_barrier();
        #endif
    } while (ways-- != 0U);
} while(sets-- != 0U);

__DSB();
__ISB();
#endif
}

```

函数描述：

此函数用于禁止数据Cache。

注意事项：

__STATIC_INLINE , __DMB , __DSB 和 __ISB 的作用看本章 2.6.1 小节的说明

2.6.6 函数 SCB_InvalidateDCache

函数原型：

```

__STATIC_INLINE void SCB_CleanDCache (void)
{
    #if defined (__DCACHE_PRESENT) && (__DCACHE_PRESENT == 1U)
        uint32_t ccsidr;
        uint32_t sets;
        uint32_t ways;

        SCB->CSSELR = 0U; /*(0U << 1U) | 0U;*/ /* Level 1 data cache */
        __DSB();

        ccsidr = SCB->CCSIDR;

        /* clean D-Cache */
        sets = (uint32_t)(CCSIDR_SETS(ccsidr));
        do {
            ways = (uint32_t)(CCSIDR_WAYS(ccsidr));
            do {
                SCB->DCCSW = (((sets << SCB_DCCSW_SET_Pos) & SCB_DCCSW_SET_Msk) |
                               ((ways << SCB_DCCSW_WAY_Pos) & SCB_DCCSW_WAY_Msk) );
                #if defined (__CC_ARM)
                    __schedule_barrier();
                #endif
            } while (ways-- != 0U);
        } while(sets-- != 0U);

        __DSB();
        __ISB();
    #endif
}

```

函数描述：

此函数用于将数据 Cache 无效化，无效化的意思是将 Cache Line 标记为无效，等同于删除操作。这样 Cache 空间就都腾出来了，可以加载新的数据。

注意事项：

__STATIC_INLINE，__DMB，__DSB 和 __ISB 的作用看本章 2.6.1 小节的说明。

2.6.7 函数 SCB_CleanDCache

函数原型：

```

__STATIC_INLINE void SCB_CleanDCache (void)
{
    #if defined (__DCACHE_PRESENT) && (__DCACHE_PRESENT == 1U)
        uint32_t ccsidr;
        uint32_t sets;
        uint32_t ways;

        SCB->CSSELR = 0U; /*(0U << 1U) | 0U;*/ /* Level 1 data cache */
        __DSB();

        ccsidr = SCB->CCSIDR;

        /* clean D-Cache */

```

```

sets = (uint32_t)(CCSIDR_SETS(ccsidr));
do {
    ways = (uint32_t)(CCSIDR_WAYS(ccsidr));
    do {
        SCB->DCCSW = (((sets << SCB_DCCSW_SET_Pos) & SCB_DCCSW_SET_Msk) |
                      ((ways << SCB_DCCSW_WAY_Pos) & SCB_DCCSW_WAY_Msk) );
        #if defined ( __CC_ARM )
            __schedule_barrier();
        #endif
    } while (ways-- != 0U);
} while(sets-- != 0U);

__DSB();
__ISB();
#endif

```

函数描述：

此函数用于将数据 Cache 清除，清除的意思是将 Cache Line 中标记为 dirty 的数据写入到相应的存储区。

注意事项：

__STATIC_INLINE，__DMB，__DSB 和 __ISB 的作用看本章 2.6.1 小节的说明。

2.6.8 函数 SCB_CleanInvalidateDCache

函数原型：

```

__STATIC_INLINE void SCB_CleanInvalidateDCache (void)
{
    #if defined ( __CC_ARM )
        uint32_t ccsidr;
    #endif
    uint32_t sets;
    uint32_t ways;

    SCB->CSSELR = 0U; /*(0U << 1U) | 0U;*/ /* Level 1 data cache */
    __DSB();

    ccsidr = SCB->CCSIDR;

    /* clean & invalidate D-Cache */
    sets = (uint32_t)(CCSIDR_SETS(ccsidr));
    do {
        ways = (uint32_t)(CCSIDR_WAYS(ccsidr));
        do {
            SCB->DCCISW = (((sets << SCB_DCCISW_SET_Pos) & SCB_DCCISW_SET_Msk) |
                          ((ways << SCB_DCCISW_WAY_Pos) & SCB_DCCISW_WAY_Msk) );
            #if defined ( __CC_ARM )
                __schedule_barrier();
            #endif
        } while (ways-- != 0U);
    } while(sets-- != 0U);

    __DSB();
    __ISB();
#endif
}

```

函数描述：

此函数是前面两个函数 SCB_InvalidateDCache 和 SCB_CleanDCache 的二合一。将 Cache Line 中标记为 dirty 的数据写入到相应的存储区后，再将 Cache Line 标记为无效，表示删除。这样 Cache 空间就都腾出来了，可以加载新的数据。

注意事项：

__STATIC_INLINE，__DMB，__DSB 和 __ISB 的作用看本章 2.6.1 小节的说明。

2.6.9 函数 SCB_InvalidateDCache_by_Addr

函数原型：

```
__STATIC_INLINE void SCB_InvalidateDCache_by_Addr (uint32_t *addr, int32_t dsize)
{
    #if defined (__DCACHE_PRESENT) && (__DCACHE_PRESENT == 1U)
        int32_t op_size = dsize;
        uint32_t op_addr = (uint32_t)addr;
        int32_t linesize = 32;          /* in Cortex-M7 size of cache line is fixed to 8 words (32 bytes) */

        __DSB();

        while (op_size > 0) {
            SCB->DCIMVAC = op_addr;
            op_addr += (uint32_t)linesize;
            op_size -= linesize;
        }

        __DSB();
        __ISB();
    #endif
}
```

函数描述：

此函数与本章 2.6.6 小节中讲解的函数作用一样，区别是这里可以指定地址和存储区大小。用于将数据 Cache 无效化，无效化的意思是将 Cache Line 标记为无效，等同于删除操作。这样 Cache 空间就都腾出来了，可以加载新的数据。

注意事项：

- ◆ __STATIC_INLINE，__DMB，__DSB 和 __ISB 的作用看本章 2.6.1 小节的说明。
- ◆ 第 1 个参数 addr：操作的地址一定要是 32 字节对齐的，即这个地址对 32 求余数等于 0。
- ◆ 第 2 个参数 dsize：一定要是 32 字节的整数倍。

2.6.10 函数 SCB_CleanDCache_by_Addr

函数原型：

```

__STATIC_INLINE void SCB_CleanDCache_by_Addr (uint32_t *addr, int32_t dsize)
{
    #if defined (__DCACHE_PRESENT) && (__DCACHE_PRESENT == 1U)
        int32_t op_size = dsize;
        uint32_t op_addr = (uint32_t) addr ;
        int32_t linesize = 32;          /* in Cortex-M7 size of cache line is fixed to 8 words (32 bytes) */

        __DSB();

        while (op_size > 0) {
            SCB->DCCMVAC = op_addr;
            op_addr += (uint32_t)linesize;
            op_size -= linesize;
        }

        __DSB();
        __ISB();
    #endif
}

```

函数描述：

此函数与本章 2.6.7 小节中讲解的函数作用一样，区别是这里可以指定地址和存储区大小。用于将数据 Cache 清除，清除的意思是将 Cache Line 中标记为 dirty 的数据写入到相应的存储区。

注意事项：

- ◆ __STATIC_INLINE，__DMB，__DSB 和 __ISB 的作用看本章 2.6.1 小节的说明。
- ◆ 第 1 个参数 addr：操作的地址一定要是 32 字节对齐的，即这个地址对 32 求余数等于 0。
- ◆ 第 2 个参数 dsize：一定要是 32 字节的整数倍。

2.6.11 函数 SCB_CleanInvalidateDCache_by_Addr

函数原型：

```

__STATIC_INLINE void SCB_CleanInvalidateDCache_by_Addr (uint32_t *addr, int32_t dsize)
{
    #if defined (__DCACHE_PRESENT) && (__DCACHE_PRESENT == 1U)
        int32_t op_size = dsize;
        uint32_t op_addr = (uint32_t) addr;
        int32_t linesize = 32;          /* in Cortex-M7 size of cache line is fixed to 8 words (32 bytes) */

        __DSB();

        while (op_size > 0) {
            SCB->DCCIMVAC = op_addr;
            op_addr += (uint32_t)linesize;
            op_size -= linesize;
        }

        __DSB();
        __ISB();
    #endif
}

```

函数描述：

此函数与本章 2.6.8 小节中讲解的函数作用一样，区别是这里可以指定地址和存储区大小。将 Cache Line 中标记为 dirty 的数据写入到相应的存储区后，再将 Cache Line 标记为无效，表示删除。这样 Cache 空间就都腾出来了，可以加载新的数据。

注意事项：

- ◆ `_STATIC_INLINE`，`_DMB`，`_DSB` 和 `_ISB` 的作用看本章 2.6.1 小节的说明。
- ◆ 第 1 个参数 `addr`：操作的地址一定要是 32 字节对齐的，即这个地址对 32 求余数等于 0。
- ◆ 第 2 个参数 `dsize`：一定要是 32 字节的整数倍。

2.7 总结

本章节就为大家讲解这么多，务必要反复多读几遍，能理解多少算多少，随着后续章节的学习再回过头来再学习。

应用举例：

测试 FMC 读写外部 SRAM，cache 的使用情况，每次先写 128 字节，再读 128 字节，循环 1000 次。配置 MPU 如下：

```

227 HAL_MPU_Disable();
228 MPU_InitStruct.Enable = MPU_REGION_ENABLE;
229 MPU_InitStruct.BaseAddress = 0x68000000;
230 MPU_InitStruct.Size = MPU_REGION_SIZE_64KB;
231 MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
232 MPU_InitStruct.IsBufferable = MPU_ACCESS_BUFFERABLE;
233 MPU_InitStruct.IsCacheable = MPU_ACCESS_CACHEABLE;
234 MPU_InitStruct.IsShareable = MPU_ACCESS_NOT_SHAREABLE;
235 MPU_InitStruct.Number = MPU_REGION_NUMBER0;
236 MPU_InitStruct.TypeExtField = MPU_TEX_LEVEL1;
237 MPU_InitStruct.SubRegionDisable = 0;
238 MPU_InitStruct.DisableExec = MPU_INSTRUCTION_ACCESS_ENABLE;
239 HAL_MPU_ConfigRegion(&MPU_InitStruct);
240 HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
241
242 HAL_MPU_Disable();
243 MPU_InitStruct.Enable = MPU_REGION_ENABLE;
244 MPU_InitStruct.BaseAddress = 0x20010000;
245 MPU_InitStruct.Size = MPU_REGION_SIZE_64KB;
246 MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
247 MPU_InitStruct.IsBufferable = MPU_ACCESS_NOT_BUFFERABLE;
248 MPU_InitStruct.IsCacheable = MPU_ACCESS_NOT_CACHEABLE;
249 MPU_InitStruct.IsShareable = MPU_ACCESS_NOT_SHAREABLE;
250 MPU_InitStruct.Number = MPU_REGION_NUMBER1;
251 MPU_InitStruct.TypeExtField = MPU_TEX_LEVEL0;
252 MPU_InitStruct.SubRegionDisable = 0;
253 MPU_InitStruct.DisableExec = MPU_INSTRUCTION_ACCESS_ENABLE;
254 HAL_MPU_ConfigRegion(&MPU_InitStruct);
255 HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);

```

0X68000000是外部FMC的映射地址，先配为TEX=1，B=1，C=1

0X20010000是存放buffer的地址，为不影响FMC的测试，将cache关闭。

代码：

```

262 HAL_GPIO_WritePin(GPIOG, GPIO_PIN_7, GPIO_PIN_SET);
263 while(cnt--)
264 {
265     printf("\r\n\r\n*****--START--%d*****\r\n\r\n", cnt);
266
267     uint16_t *p_write = (uint16_t *)0x20010000;
268     uint16_t *p_read = (uint16_t *)0x20014000;
269     for(uint32_t i = 0; i < BUFFERSIZE; i++)
270     {
271         *p_write++ = i + cnt;
272     }
273
274     for(uint32_t i = 0; i < BUFFERSIZE; i++)
275     {
276         //
277         //
278     }
279     p_write = (uint16_t *)0x20010000;
280     HAL_SRAM_Write_16b(&SramHandle, (uint32_t*)(SRAM_ADDR), p_write, BUFFERSIZE);
281     // SCB_CleanDCache_by_Addr((uint32_t*)(SRAM_ADDR), BUFFERSIZE);
282
283     HAL_SRAM_Read_16b(&SramHandle, (uint32_t*)(SRAM_ADDR), p_read, BUFFERSIZE);
291     p_write = (uint16_t *)0x20010000;
292     p_read = (uint16_t *)0x20014000;
293     compare_buff(p_write, p_read, BUFFERSIZE);

```

结果：

可以看出，写操作并没有在FMC上实际有效。

修改代码:

```
260 void fmc_sram_test(uint32_t cnt)
261 {
262     HAL_GPIO_WritePin(GPIOG, GPIO_PIN_7, GPIO_PIN_SET);
263     while(cnt-->0)
264     {
265         printf("\r\n\r\n*****--START--%d*****\r\n\r\n", cnt);
266
267         uint16_t *p_write = (uint16_t *)0x20010000;
268         uint16_t *p_read = (uint16_t *)0x20014000;
269         for(uint32_t i = 0; i < BUFFERSIZE; i++)
270         {
271             *p_write++ = i + cnt;
272         }
273
274         // for(uint32_t i = 0; i < BUFFERSIZE; i++)
275         // {
276         //
277         // }
278
279         p_write = (uint16_t *)0x20010000;
280         HAL_SRAM_Write_16b(&SramHandle, (uint32_t *) (SRAM_ADDR), p_write, BUFFERSIZE);
281         SCB_CleanDCache_by_Addr ((uint32_t *) (SRAM_ADDR), BUFFERSIZE);
282
283         SCB_InvalidateDCache_by_Addr ((uint32_t *) (SRAM_ADDR), BUFFERSIZE);
284         HAL_SRAM_Read_16b(&SramHandle, (uint32_t *) (SRAM_ADDR), p_read, BUFFERSIZE);
291         p_write = (uint16_t *)0x20010000;
292         p_read = (uint16_t *)0x20014000;
293         compare_buff(p_write, p_read, BUFFERSIZE);
294         // Print_HalfWordBuff("p_write", p_write, BUFFERSIZE);
295         // Print_HalfWordBuff("p_read", p_read, BUFFERSIZE);
296
297
298     }
299     HAL_GPIO_WritePin(GPIOG, GPIO_PIN_7, GPIO_PIN_RESET);
```

可以看到，每次写完以后，对cache进行clean操作，将cache的数据写入到0X68000000，每次读之前，将cache invalid，再读的话会将0X68000000读到cache，再将cache读到内核。该配置有一个问题，第一次写的时候会先将0X68000000的值读到cache，再写cache，这就会在FMC总线上产生一次读操作

测试RAM搬数到RAM

配置MPU:

```

void mpu_config(void)
{
    MPU_Region_InitTypeDef MPU_InitStruct;

    HAL_MPU_Disable();
    MPU_InitStruct.Enable = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress = 0x20030000;
    MPU_InitStruct.Size = MPU_REGION_SIZE_16KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable = MPU_ACCESS_BUFFERABLE;
    MPU_InitStruct.IsCacheable = MPU_ACCESS_CACHEABLE;
    MPU_InitStruct.IsShareable = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number = MPU_REGION_NUMBER0;
    MPU_InitStruct.TypeExtField = MPU_TEX_LEVEL1;
    MPU_InitStruct.SubRegionDisable = 0;
    MPU_InitStruct.DisableExec = MPU_INSTRUCTION_ACCESS_ENABLE;
    HAL_MPU_ConfigRegion(&MPU_InitStruct);
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);

    HAL_MPU_Disable();
    MPU_InitStruct.Enable = MPU_REGION_ENABLE;
    MPU_InitStruct.BaseAddress = 0x20040000;
    MPU_InitStruct.Size = MPU_REGION_SIZE_32KB;
    MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
    MPU_InitStruct.IsBufferable = MPU_ACCESS_NOT_BUFFERABLE;
    MPU_InitStruct.IsCacheable = MPU_ACCESS_NOT_CACHEABLE;
    MPU_InitStruct.IsShareable = MPU_ACCESS_NOT_SHAREABLE;
    MPU_InitStruct.Number = MPU_REGION_NUMBER1;
    MPU_InitStruct.TypeExtField = MPU_TEX_LEVEL0;
    MPU_InitStruct.SubRegionDisable = 0;
    MPU_InitStruct.DisableExec = MPU_INSTRUCTION_ACCESS_ENABLE;
    HAL_MPU_ConfigRegion(&MPU_InitStruct);
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}

```

配置2块RAM地址，0x20030000配为，TEX=1，C=1，B=1，write back，write and read allocate。
0x20040000配置为TEX=0，B=0，C=0，no cache模式。

1.配置代码如下：

```

268 void ram1_to_ram2(uint32_t cnt)
269 {
270     printf("\r\n\r\n*****--START--*****\r\n\r\n");
271     uint32_t *p_print;
272     HAL_GPIO_WritePin(GPIOG, GPIO_PIN_12, GPIO_PIN_SET);
273
274     uint32_t *p = (uint32_t *)0x20040000;
275     for(uint32_t i = 0; i < 10; i++)
276     {
277         *p++ = i + 0xAAAABBBB + cnt;
278     }
279
312     HAL_DMA_Start(&DMA_MTMWordHandle, (uint32_t)0x20040000, 0x20030000, 1*10);
313     HAL_DMA_PollForTransfer(&DMA_MTMWordHandle, HAL_DMA_FULL_TRANSFER, HAL_SRAM_DMA_TIMEOUT);
314
315     // SCB_InvalidateDCache_by_Addr ((uint32_t *)0x20030000, 4*10);
316     p_print = (uint32_t *)0x20030000;
317     Print_WordBuff("0x20030000", p_print, 10);
318
334     p_print = (uint32_t *)0x20040000;
335     Print_WordBuff("0x20040000", p_print, 10);
}

```

执行结果：

```

*****--START--*****

buffer: 0x20030000, len 10
{
AAAABBBB AAAABBBC AAAABBD AAAABBE AAAABBF AAAABBC0 AAAABBC1 AAAABBC2 AAAABBC3 AAAABBC4
}
buffer: 0x20040000, len 10
{
AAAABBBB AAAABBBC AAAABBD AAAABBE AAAABBF AAAABBC0 AAAABBC1 AAAABBC2 AAAABBC3 AAAABBC4
}

*****--START--*****

buffer: 0x20030000, len 10
{
AAAABBBB AAAABBBC AAAABBD AAAABBE AAAABBF AAAABBC0 AAAABBC1 AAAABBC2 AAAABBC3 AAAABBC4
}
buffer: 0x20040000, len 10
{
AAAABBBC AAAABBD AAAABBE AAAABBF AAAABBC0 AAAABBC1 AAAABBC2 AAAABBC3 AAAABBC4 AAAABBC5
}

*****--START--*****

buffer: 0x20030000, len 10
{
AAAABBBB AAAABBBC AAAABBD AAAABBE AAAABBF AAAABBC0 AAAABBC1 AAAABBC2 AAAABBC3 AAAABBC4
}
buffer: 0x20040000, len 10
{
AAAABBD AAAABBE AAAABBF AAAABBC0 AAAABBC1 AAAABBC2 AAAABBC3 AAAABBC4 AAAABBC5 AAAABBC6
}

```

代码更改如下：

```

312 HAL_DMA_Start(&DMA_MTMWordHandle, (uint32_t)0x20040000, 0x20030000, 1*10);
313 HAL_DMA_PollForTransfer(&DMA_MTMWordHandle, HAL_DMA_FULL_TRANSFER, HAL_SRAM_DMA_TIMEOUT);
314
315 SCB_InvalidateDCache_by_Addr ((uint32_t *)0x20030000, 4*10);
316 p_print = (uint32_t *)0x20030000;
317 Print_WordBuff("0x20030000", p_print, 10);

```

加了一行315代码，执行结果如下：

```

*****--START--*****

buffer: 0x20030000, len 10
{
AAAABBBB AAAABBBC AAAABBBB AAAABBBE AAAABBBF AAAABBC0 AAAABBC1 AAAABBC2 AAAABBC3 AAAABBC4
}
buffer: 0x20040000, len 10
{
AAAABBBB AAAABBBC AAAABBBB AAAABBBE AAAABBBF AAAABBC0 AAAABBC1 AAAABBC2 AAAABBC3 AAAABBC4
}

*****--START--*****

buffer: 0x20030000, len 10
{
AAAABBBC AAAABBBB AAAABBBE AAAABBBF AAAABBC0 AAAABBC1 AAAABBC2 AAAABBC3 AAAABBC4 AAAABBC5
}
buffer: 0x20040000, len 10
{
AAAABBBC AAAABBBB AAAABBBE AAAABBBF AAAABBC0 AAAABBC1 AAAABBC2 AAAABBC3 AAAABBC4 AAAABBC5
}

*****--START--*****

buffer: 0x20030000, len 10
{
AAAABBBB AAAABBBE AAAABBBF AAAABBC0 AAAABBC1 AAAABBC2 AAAABBC3 AAAABBC4 AAAABBC5 AAAABBC6
}
buffer: 0x20040000, len 10
{
AAAABBBB AAAABBBE AAAABBBF AAAABBC0 AAAABBC1 AAAABBC2 AAAABBC3 AAAABBC4 AAAABBC5 AAAABBC6
}

```

```

295 uint32_t *p1 = (uint32_t *)0x20030000;
296 for(uint32_t i = 0; i < 10; i++)
297 {
298     *p1++ = i + 0x11220000 + cnt;
299 }
300 p_print = (uint32_t *)0x20030000;
301 Print_WordBuff("0x20030000", p_print, 10);
302 // SCB_CleanDCache_by_Addr((uint32_t *)0x20030000, 4*256);
303
304 p1 = (uint32_t *)0x20030000;
305 for(uint32_t i = 0; i < 10; i++)
306 {
307     *p1++ = i + 0x33440000 + cnt;
308 }
309 p_print = (uint32_t *)0x20030000;
310 Print_WordBuff("0x20030000", p_print, 10);

```

```

321 HAL_DMA_Start(&DMA_MTMWordHandle, (uint32_t)0x20030000, 0x20041000, 1*10);
322 HAL_DMA_PollForTransfer(&DMA_MTMWordHandle, HAL_DMA_FULL_TRANSFER, HAL_SRAM_DMA_TIMEOUT);
323 // SCB_InvalidateDCache_by_Addr((uint32_t*)0x20030000, 4*10);
324 //
325 // memcpy((uint32_t*)0x20041000, (uint32_t*)0x20030000, 4*10);
326 //
327 // p1 = (uint32_t*)0x20030000;
328 // uint32_t *p2 = (uint32_t *)0x20041000;
329 // for(uint32_t i = 0; i < 256; i++)
330 // {
331 //     *p2++ = *p1++;
332 // }
333
334 // p_print = (uint32_t*)0x20040000;
335 // Print_WordBuff("0x20040000", p_print, 10);
336 p_print = (uint32_t*)0x20041000;
337 Print_WordBuff("0x20041000", p_print, 10);

```

结果:

```

*****--START--*****

buffer: 0x20030000, len 10
{
11220000 11220001 11220002 11220003 11220004 11220005 11220006 11220007 11220008 11220009
}
buffer: 0x20030000, len 10
{
33440000 33440001 33440002 33440003 33440004 33440005 33440006 33440007 33440008 33440009
}
buffer: 0x20041000, len 10
{
3FE017EE D07393B6 C9D15C2F 43838602 FCF1FE76 261EEFEF 23B7F328 F7B277E8 63BBE9DE 688509EC
}

*****--START--*****

buffer: 0x20030000, len 10
{
11220001 11220002 11220003 11220004 11220005 11220006 11220007 11220008 11220009 1122000A
}
buffer: 0x20030000, len 10
{
33440001 33440002 33440003 33440004 33440005 33440006 33440007 33440008 33440009 3344000A
}
buffer: 0x20041000, len 10
{
3FE017EE D07393B6 C9D15C2F 43838602 FCF1FE76 261EEFEF 23B7F328 F7B277E8 63BBE9DE 688509EC
}

```

更改代码:

```

295 uint32_t *p1 = (uint32_t *)0x20030000;
296 for(uint32_t i = 0; i < 10; i++)
297 {
298     *p1++ = i + 0x11220000 + cnt;
299 }
300 p_print = (uint32_t *)0x20030000;
301 Print_WordBuff("0x20030000", p_print, 10);
302 // SCB_CleanDCache_by_Addr((uint32_t *)0x20030000, 4*256);
303
304 p1 = (uint32_t *)0x20030000;
305 for(uint32_t i = 0; i < 10; i++)
306 {
307     *p1++ = i + 0x33440000 + cnt;
308 }
309 p_print = (uint32_t *)0x20030000;
310 Print_WordBuff("0x20030000", p_print, 10);
319 SCB_CleanDCache_by_Addr((uint32_t *)0x20030000, 4*10);
320 //
321 HAL_DMA_Start(&DMA_MTMWordHandle, (uint32_t)0x20030000, 0x20041000, 1*10);
322 HAL_DMA_PollForTransfer(&DMA_MTMWordHandle, HAL_DMA_FULL_TRANSFER, HAL_SRAM_DMA_TIMEOUT);
336 p_print = (uint32_t *)0x20041000;
337 Print_WordBuff("0x20041000", p_print, 10);

```

结果:

```

*****--START--*****
buffer: 0x20030000, len 10
{
11220000 11220001 11220002 11220003 11220004 11220005 11220006 11220007 11220008 11220009
}
buffer: 0x20030000, len 10
{
33440000 33440001 33440002 33440003 33440004 33440005 33440006 33440007 33440008 33440009
}
buffer: 0x20041000, len 10
{
33440000 33440001 33440002 33440003 33440004 33440005 33440006 33440007 33440008 33440009
}

*****--START--*****
buffer: 0x20030000, len 10
{
11220001 11220002 11220003 11220004 11220005 11220006 11220007 11220008 11220009 1122000A
}
buffer: 0x20030000, len 10
{
33440001 33440002 33440003 33440004 33440005 33440006 33440007 33440008 33440009 3344000A
}
buffer: 0x20041000, len 10
{
33440001 33440002 33440003 33440004 33440005 33440006 33440007 33440008 33440009 3344000A
}

```

更改代码:

```

295 uint32_t *p1 = (uint32_t *)0x20030000;
296 for(uint32_t i = 0; i < 10; i++)
297 {
298     *p1++ = i + 0x11220000 + cnt;
299 }
300 p_print = (uint32_t *)0x20030000;
301 Print_WordBuff("0x20030000", p_print, 10);
302 // SCB_CleanDCache_by_Addr((uint32_t *)0x20030000, 4*256);
303
304 p1 = (uint32_t *)0x20030000;
305 for(uint32_t i = 0; i < 10; i++)
306 {
307     *p1++ = i + 0x33440000 + cnt;
308 }
309 p_print = (uint32_t *)0x20030000;
310 Print_WordBuff("0x20030000", p_print, 10);
311
312 p1 = (uint32_t *)0x20030000;
313 uint32_t *p2 = (uint32_t *)0x20041000;
314 for(uint32_t i = 0; i < 256; i++)
315 {
316     *p2++ = *p1++;
317 }
318
319 // p_print = (uint32_t *)0x20040000;
320 // Print_WordBuff("0x20040000", p_print, 10);
321 p_print = (uint32_t *)0x20041000;
322 Print_WordBuff("0x20041000", p_print, 10);
323
324
325

```

结果:

```

*****--START--*****

buffer: 0x20030000, len 10
{
11220000 11220001 11220002 11220003 11220004 11220005 11220006 11220007 11220008 11220009
}
buffer: 0x20030000, len 10
{
33440000 33440001 33440002 33440003 33440004 33440005 33440006 33440007 33440008 33440009
}
buffer: 0x20041000, len 10
{
33440000 33440001 33440002 33440003 33440004 33440005 33440006 33440007 33440008 33440009
}

*****--START--*****

buffer: 0x20030000, len 10
{
11220001 11220002 11220003 11220004 11220005 11220006 11220007 11220008 11220009 1122000A
}
buffer: 0x20030000, len 10
{
33440001 33440002 33440003 33440004 33440005 33440006 33440007 33440008 33440009 3344000A
}
buffer: 0x20041000, len 10
{
33440001 33440002 33440003 33440004 33440005 33440006 33440007 33440008 33440009 3344000A
}

```

```

325 memcpy((uint32_t *)0x20041000, (uint32_t *)0x20030000, 4*10);
326
327 // p1 = (uint32_t *)0x20030000;
328 // uint32_t *p2 = (uint32_t *)0x20041000;
329 // for(uint32_t i = 0; i < 256; i++)
330 // {
331 //     *p2++ = *p1++;
332 // }

```

通过memcpy也是一样的效果。

测试RAM搬数到RAM

配置MPU:

```

208 HAL_MPU_Disable();
209 MPU_InitStruct.Enable = MPU_REGION_ENABLE;
210 MPU_InitStruct.BaseAddress = 0x20030000;
211 MPU_InitStruct.Size = MPU_REGION_SIZE_16KB;
212 MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
213 MPU_InitStruct.IsBufferable = MPU_ACCESS_NOT_BUFFERABLE;
214 MPU_InitStruct.IsCacheable = MPU_ACCESS_CACHEABLE;
215 MPU_InitStruct.IsShareable = MPU_ACCESS_NOT_SHAREABLE;
216 MPU_InitStruct.Number = MPU_REGION_NUMBER0;
217 MPU_InitStruct.TypeExtField = MPU_TEX_LEVEL0;
218 MPU_InitStruct.SubRegionDisable = 0;
219 MPU_InitStruct.DisableExec = MPU_INSTRUCTION_ACCESS_ENABLE;
220 HAL_MPU_ConfigRegion(&MPU_InitStruct);
221 HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
222
223 HAL_MPU_Disable();
224 MPU_InitStruct.Enable = MPU_REGION_ENABLE;
225 MPU_InitStruct.BaseAddress = 0x20040000;
226 MPU_InitStruct.Size = MPU_REGION_SIZE_32KB;
227 MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
228 MPU_InitStruct.IsBufferable = MPU_ACCESS_NOT_BUFFERABLE;
229 MPU_InitStruct.IsCacheable = MPU_ACCESS_NOT_CACHEABLE;
230 MPU_InitStruct.IsShareable = MPU_ACCESS_NOT_SHAREABLE;
231 MPU_InitStruct.Number = MPU_REGION_NUMBER1;
232 MPU_InitStruct.TypeExtField = MPU_TEX_LEVEL0;
233 MPU_InitStruct.SubRegionDisable = 0;
234 MPU_InitStruct.DisableExec = MPU_INSTRUCTION_ACCESS_ENABLE;
235 HAL_MPU_ConfigRegion(&MPU_InitStruct);
236 HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);

```

0X20030000配置为TXE=0, B=0, C=1, write through, no write allocate, read allocate

0X20040000配置为TXE=0, B=0, C=0, no cache模式

第一种情况:

```

243 printf("\r\n\r\n*****--START--*****\r\n\r\n");
244 uint32_t *p_print;
245 HAL_GPIO_WritePin(GPIOG, GPIO_PIN_12, GPIO_PIN_SET);
246
247 uint32_t *p = (uint32_t *)0x20040000;
248 for(uint32_t i = 0; i < 10; i++)
249 {
250     *p++ = i + 0x11223344 + cnt;
251 }
252 p_print = (uint32_t *)0x20040000;
253 Print_WordBuff("0x20040000", p_print, 10);
254
255 HAL_DMA_Start(&DMA_MTMWordHandle, (uint32_t)0x20040000, 0x20030000, 1*10);
256 HAL_DMA_PollForTransfer(&DMA_MTMWordHandle, HAL_DMA_FULL_TRANSFER, HAL_SRAM_DMA_TIMEOUT);
257
258 p_print = (uint32_t *)0x20030000;
259 Print_WordBuff("0x20030000", p_print, 10);

```

结果:

```

buffer: 0x20040000, len 10
{
11223344 11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D
}
buffer: 0x20030000, len 10
{
11223344 11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D
}

*****--START--*****

buffer: 0x20040000, len 10
{
11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E
}
buffer: 0x20030000, len 10
{
11223344 11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D
}

*****--START--*****

buffer: 0x20040000, len 10
{
11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E 1122334F
}
buffer: 0x20030000, len 10
{
11223344 11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D
}

```

修改代码:

```

243 printf("\r\n\r\n*****--START--*****\r\n\r\n");
244 uint32_t *p_print;
245 HAL_GPIO_WritePin(GPIOG, GPIO_PIN_12, GPIO_PIN_SET);
246
247 uint32_t *p = (uint32_t *)0x20040000;
248 for(uint32_t i = 0; i < 10; i++)
249 {
250     *p++ = i + 0x11223344 + cnt;
251 }
252 p_print = (uint32_t *)0x20040000;
253 Print_WordBuff("0x20040000", p_print, 10);
254
255 SCB_InvalidateDCache_by_Addr((uint32_t *)0x20030000, 4*10);
256 HAL_DMA_Start(&DMA_MTMWordHandle, (uint32_t)0x20040000, 0x20030000, 1*10);
257 HAL_DMA_PollForTransfer(&DMA_MTMWordHandle, HAL_DMA_FULL_TRANSFER, HAL_SRAM_DMA_TIMEOUT);
258
259 p_print = (uint32_t *)0x20030000;
260 Print_WordBuff("0x20030000", p_print, 10);

```

结果:

```

buffer: 0x20040000, len 10
{
11223344 11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D
}
buffer: 0x20030000, len 10
{
11223344 11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D
}

*****--START--*****

buffer: 0x20040000, len 10
{
11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E
}
buffer: 0x20030000, len 10
{
11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E
}

*****--START--*****

buffer: 0x20040000, len 10
{
11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E 1122334F
}
buffer: 0x20030000, len 10
{
11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E 1122334F
}

```

修改代码:

```

243 printf("\r\n\r\n*****--START--*****\r\n\r\n");
244 uint32_t *p_print;
245 HAL_GPIO_WritePin(GPIOG, GPIO_PIN_12, GPIO_PIN_SET);
246
247 uint32_t *p = (uint32_t *)0x20040000;
248 for(uint32_t i = 0; i < 10; i++)
249 {
250     *p++ = i + 0x11223344 + cnt;
251 }
252 p_print = (uint32_t *)0x20040000;
253 Print_WordBuff("0x20040000", p_print, 10);
254
255 uint32_t tmp;
256 p = (uint32_t *)0x20030000;
257 for(uint32_t i = 0; i < 10; i++)
258 {
259     tmp = *p++;
260 }
261
262 // SCB_InvalidateDCache_by_Addr((uint32_t *)0x20030000, 4*10);
263 HAL_DMA_Start(&DMA_MTMWordHandle, (uint32_t)0x20040000, 0x20030000, 1*10);
264 HAL_DMA_PollForTransfer(&DMA_MTMWordHandle, HAL_DMA_FULL_TRANSFER, HAL_SRAM_DMA_TIMEOUT);
265
266 p_print = (uint32_t *)0x20030000;
267 Print_WordBuff("0x20030000", p_print, 10);
268

```

结果:

```

buffer: 0x20040000, len 10
{
11223344 11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D
}
buffer: 0x20030000, len 10
{
46CF5F21 A229EA1B 8D014E18 8DB8D52C 83004105 DEEE3F55 181CD9DB 105B55C8 D89EC73E E6A72C8B
}

*****--START--*****

buffer: 0x20040000, len 10
{
11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E
}
buffer: 0x20030000, len 10
{
46CF5F21 A229EA1B 8D014E18 8DB8D52C 83004105 DEEE3F55 181CD9DB 105B55C8 D89EC73E E6A72C8B
}

*****--START--*****

buffer: 0x20040000, len 10
{
11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E 1122334F
}
buffer: 0x20030000, len 10
{
46CF5F21 A229EA1B 8D014E18 8DB8D52C 83004105 DEEE3F55 181CD9DB 105B55C8 D89EC73E E6A72C8B
}

```

```

243 printf("\r\n\r\n*****--START--*****\r\n\r\n");
244 uint32_t *p_print;
245 HAL_GPIO_WritePin(GPIOG, GPIO_PIN_12, GPIO_PIN_SET);
246
247 uint32_t *p = (uint32_t *)0x20040000;
248 for(uint32_t i = 0; i < 10; i++)
249 {
250     *p++ = i + 0x11223344 + cnt;
251 }
252 p_print = (uint32_t *)0x20040000;
253 Print_WordBuff("0x20040000", p_print, 10);
254
255 uint32_t tmp;
256 p = (uint32_t *)0x20030000;
257 for(uint32_t i = 0; i < 10; i++)
258 {
259     tmp = *p++;
260 }
261
262 SCB_InvalidateDCache_by_Addr((uint32_t *)0x20030000, 4*10);
263 HAL_DMA_Start(&DMA_MTMWordHandle, (uint32_t)0x20040000, 0x20030000, 1*10);
264 HAL_DMA_PollForTransfer(&DMA_MTMWordHandle, HAL_DMA_FULL_TRANSFER, HAL_SRAM_DMA_TIMEOUT);
265
266 p_print = (uint32_t *)0x20030000;
267 Print_WordBuff("0x20030000", p_print, 10);

```

```

F buffer: 0x20040000, len 10
{
11223344 11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D
}
buffer: 0x20030000, len 10
{
11223344 11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D
}

*****--START--*****

buffer: 0x20040000, len 10
{
11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E
}
buffer: 0x20030000, len 10
{
11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E
}

*****--START--*****

buffer: 0x20040000, len 10
{
11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E 1122334F
}
buffer: 0x20030000, len 10
{
11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E 1122334F
}

```

修改代码:

```

243 printf("\r\n\r\n*****--START--*****\r\n\r\n");
244 uint32_t *p_print;
245 HAL_GPIO_WritePin(GPIOG, GPIO_PIN_12, GPIO_PIN_SET);
246
247 uint32_t *p = (uint32_t *)0x20040000;
248 for(uint32_t i = 0; i < 10; i++)
249 {
250     *p++ = i + 0x11223344 + cnt;
251 }
252 p_print = (uint32_t *)0x20040000;
253 Print_WordBuff("0x20040000", p_print, 10);
254
255 // uint32_t tmp;
256 p = (uint32_t *)0x20030000;
257 for(uint32_t i = 0; i < 10; i++)
258 {
259     *p++ = i + 0xABCD0000 + cnt;
260 }
261
262 // SCB_InvalidateDCache_by_Addr((uint32_t *)0x20030000, 4*10);
263 HAL_DMA_Start(&DMA_MTMWordHandle, (uint32_t)0x20040000, 0x20030000, 1*10);
264 HAL_DMA_PollForTransfer(&DMA_MTMWordHandle, HAL_DMA_FULL_TRANSFER, HAL_SRAM_DMA_TIMEOUT);
265
266 p_print = (uint32_t *)0x20030000;
267 Print_WordBuff("0x20030000", p_print, 10);

```

```

buffer: 0x20040000, len 10
{
11223344 11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D
}
buffer: 0x20030000, len 10
{
11223344 11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D
}

*****--START--*****

buffer: 0x20040000, len 10
{
11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E
}
buffer: 0x20030000, len 10
{
ABCD0001 ABCD0002 ABCD0003 ABCD0004 ABCD0005 ABCD0006 ABCD0007 ABCD0008 ABCD0009 ABCD000A
}

*****--START--*****

buffer: 0x20040000, len 10
{
11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E 1122334F
}
buffer: 0x20030000, len 10
{
ABCD0002 ABCD0003 ABCD0004 ABCD0005 ABCD0006 ABCD0007 ABCD0008 ABCD0009 ABCD000A ABCD000B
}

```

修改代码:

```

242 {
243     printf("\r\n\r\n*****--START--*****\r\n\r\n");
244     uint32_t *p_print;
245     HAL_GPIO_WritePin(GPIOG, GPIO_PIN_12, GPIO_PIN_SET);
246
247     uint32_t *p = (uint32_t *)0x20040000;
248     for(uint32_t i = 0; i < 10; i++)
249     {
250         *p++ = i + 0x11223344 + cnt;
251     }
252     p_print = (uint32_t *)0x20040000;
253     Print_WordBuff("0x20040000", p_print, 10);
254
255     // uint32_t tmp;
256     p = (uint32_t *)0x20030000;
257     for(uint32_t i = 0; i < 10; i++)
258     {
259         *p++ = i + 0xABCD0000 + cnt;
260     }
261
262     SCB_InvalidateDCache_by_Addr ((uint32_t *)0x20030000, 4*10);
263     HAL_DMA_Start(&DMA_MTMWordHandle, (uint32_t)0x20040000, 0x20030000, 1*10);
264     HAL_DMA_PollForTransfer(&DMA_MTMWordHandle, HAL_DMA_FULL_TRANSFER, HAL_SRAM_DMA_TIMEOUT);
265
266     p_print = (uint32_t *)0x20030000;
267     Print_WordBuff("0x20030000", p_print, 10);
268
269

```

```

buffer: 0x20040000, len 10
{
11223344 11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D
}
buffer: 0x20030000, len 10
{
11223344 11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D
}

*****--START--*****

buffer: 0x20040000, len 10
{
11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E
}
buffer: 0x20030000, len 10
{
11223345 11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E
}

*****--START--*****

buffer: 0x20040000, len 10
{
11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E 1122334F
}
buffer: 0x20030000, len 10
{
11223346 11223347 11223348 11223349 1122334A 1122334B 1122334C 1122334D 1122334E 1122334F
}

```

加了262行无效化后，后面的读取都是最新的SRAM内部的数据

结论：

1. 将某块SRAM配置为配置为TXE=0, B=0, C=1, write through, no write allocate, read allocate的时候，写：直接写某一块地址，如果cache已经命中了，此时写操作会同时更新cache和SRAM的内容，如果miss，则会只写SRAM而不更新cache。注意，写操作，不会使cache命中。

读：第一次读cache是miss，此时读了以后cache就命中了；cache miss时，读操作会先将SRAM中的数更新到cache，再读取cache的数到SRAM；cache命中的时候，会直接读取cache的数据到内核

串口DMA+IDLE+MPU配置说明：

```

HAL_MPU_Disable();
MPU_InitStruct.Enable = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress = 0x20000000;
MPU_InitStruct.Size = MPU_REGION_SIZE_64KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable = MPU_ACCESS_BUFFERABLE;
MPU_InitStruct.IsCacheable = MPU_ACCESS_CACHEABLE;
MPU_InitStruct.IsShareable = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number = MPU_REGION_NUMBER0;
MPU_InitStruct.TypeExtField = MPU_TEX_LEVEL0;
MPU_InitStruct.SubRegionDisable = 0;
MPU_InitStruct.DisableExec = MPU_INSTRUCTION_ACCESS_ENABLE;
HAL_MPU_ConfigRegion(&MPU_InitStruct);
HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);

__HAL_UART_ENABLE_IT(&huart_console, UART_IT_IDLE);

HAL_UART_Receive_DMA(&huart_console, RX_BUFFER, UART_BUFFER_SIZE);
SCB_CleanInvalidateDCache_by_Addr((uint32_t *)RX_BUFFER, UART_BUFFER_SIZE);

```

```

5 void USART1_IRQHandler(void)
6 {
7     if (CONSOLE_INSTANCE == huart_console.Instance)
8     {
9         if (RESET != __HAL_UART_GET_FLAG(&huart_console, UART_FLAG_IDLE))
10        {
11            __HAL_UART_CLEAR_IDLEFLAG(&huart_console);
12            HAL_UART_DMAStop(&huart_console);
13            uint16_t recv = UART_BUFFER_SIZE - __HAL_DMA_GET_COUNTER(&hdma_console_rx);
14            Print_ByteBuff("RX_BUFFER", RX_BUFFER, recv);
15            HAL_UART_Receive_DMA(&huart_console, RX_BUFFER, UART_BUFFER_SIZE);
16            SCB_CleanInvalidateDCache_by_Addr((uint32_t*)RX_BUFFER, UART_BUFFER_SIZE);
17        }
18    }
19    HAL_UART_IRQHandler(&huart_console);
20 }

```